

Day 4 — Production Python Concurrency, Testing, Observability, and Resilience

Beginner-friendly summary

Production AI services spend much of their time waiting for LLM providers, vector databases, object storage, APIs, and relational databases. `asyncio` lets one Python thread work on other requests during those waits.

The central rule is:

| Use `async` to overlap waiting, not to accelerate CPU-heavy Python computation.

A production implementation should also:

- Limit concurrency instead of creating unlimited tasks.
- Put deadlines around every external operation.
- Retry only explicitly transient failures.
- Propagate cancellation instead of swallowing it.
- Apply backpressure when producers are faster than consumers.
- Inject external dependencies so tests remain deterministic.
- Log identifiers and outcomes, but not prompts, documents, credentials, or personal data.

The practical implementation below uses Python 3.11+ because `asyncio.timeout()` was added in Python 3.11. It internally uses cancellation and converts the cancellation into `TimeoutError` outside its context. ([Python documentation](#))

1. Choosing the concurrency model

Model	Best fit	Avoid when	Selection criterion
Synchronous	Simple scripts, short request flows, low concurrency	Many independent network waits	Choose when simplicity matters more than throughput
<code>asyncio</code>	LLM calls, HTTP, async DB drivers, retrieval, object storage	CPU-heavy Python loops or blocking libraries	Choose when most time is spent awaiting async-compatible I/O
Threads	Blocking SDKs, legacy DB drivers, file operations	Pure-Python CPU-heavy work	Choose when the dependency blocks and cannot be made async
Processes	Parsing, image transformation, CPU-heavy feature extraction	Small tasks with large serialization overhead	Choose when Python CPU work must use multiple cores
External worker system	Long-running ingestion, durable retries, scheduled jobs	Small request-local fan-out	Choose when work must survive process or machine failure

In standard CPython builds, the GIL normally permits only one thread to execute Python bytecode at a time. Threads are therefore most useful for overlapping I/O, while processes are generally more suitable for CPU-bound Python work. Free-threaded builds exist but are not the default assumption for production architecture. ([Python documentation](#))

Interview answer

I choose concurrency based on the dominant bottleneck. Async is my default for high-volume network I/O when the entire dependency chain is async-compatible. I use a bounded thread pool for blocking SDKs and a process pool or separate worker service for CPU-heavy Python. I do not add async merely because the service is an AI service.

2. Synchronous versus asynchronous execution

Synchronous

A synchronous function keeps control until it finishes:

```
result_a = call_retriever()
result_b = call_metadata_service()
```

If each operation waits one second, the total is roughly two seconds.

Asynchronous

Independent operations can overlap:

```
result_a, result_b = await asyncio.gather(
    call_retriever(),
    call_metadata_service(),
)
```

The elapsed time approaches the slower operation rather than their sum, assuming both are truly asynchronous and there is sufficient downstream capacity.

I/O-bound work

Typical I/O-bound operations include:

- LLM API calls
- Vector database searches
- PostgreSQL queries through an async driver
- Redis calls
- S3 or object-storage operations
- Tool or microservice HTTP calls
- Reading network-mounted files

CPU-bound work

Typical CPU-bound operations include:

- Large PDF parsing in pure Python
- Image preprocessing
- Tokenization or feature extraction that does not release the GIL
- Large JSON transformations
- Local model inference
- Numerical loops written in Python

Some native libraries release the GIL, so measurement is more reliable than assuming every numerical operation is blocked by it.

3. Understanding the event loop

The event loop executes one runnable coroutine at a time in its thread. A coroutine cooperatively yields control when it reaches an `await` whose operation is not ready.

```
async def fetch_document(document_id: str) -> bytes:
    return await storage_client.download(document_id)
```

Calling `fetch_document()` does not immediately execute it to completion. It creates a coroutine object. Execution occurs when it is awaited or scheduled as a task.

```
task = asyncio.create_task(fetch_document("doc-123"))
document = await task
```

Task

A task is a scheduled coroutine with lifecycle state:

- Pending
- Running
- Completed
- Failed
- Cancelled

Always retain or await tasks you create. Fire-and-forget tasks commonly cause:

- Lost exceptions
 - Premature garbage collection
 - Incomplete work during shutdown
 - Resource leaks
 - Requests continuing after clients disconnect
-

4. gather, semaphores, queues, and bounded concurrency

`asyncio.gather`

Use `gather` when you have a known, reasonably small group of independent operations:

```
chunks, metadata = await asyncio.gather(
    retriever.search(query),
    metadata_client.lookup(customer_id),
)
```

`gather` coordinates completion, but it does **not** limit the number of operations started.

This is dangerous:

```
await asyncio.gather(
    *(process(document) for document in one_million_documents)
)
```

It can create a very large number of tasks and consume excessive memory before downstream calls even begin.

Semaphore

A semaphore limits how many coroutines enter a protected section:

```
semaphore = asyncio.Semaphore(10)

async def bounded_call(document):
    async with semaphore:
        return await provider.process(document)
```

A semaphore decrements its counter when acquired and waits when the counter reaches zero. The `async with` form ensures release even when an exception occurs. ([Python documentation](#))

However, one task per input can still create a large number of waiting tasks.

Bounded queue

A bounded queue controls both:

1. The number of active consumers.
2. The number of pending items held in memory.

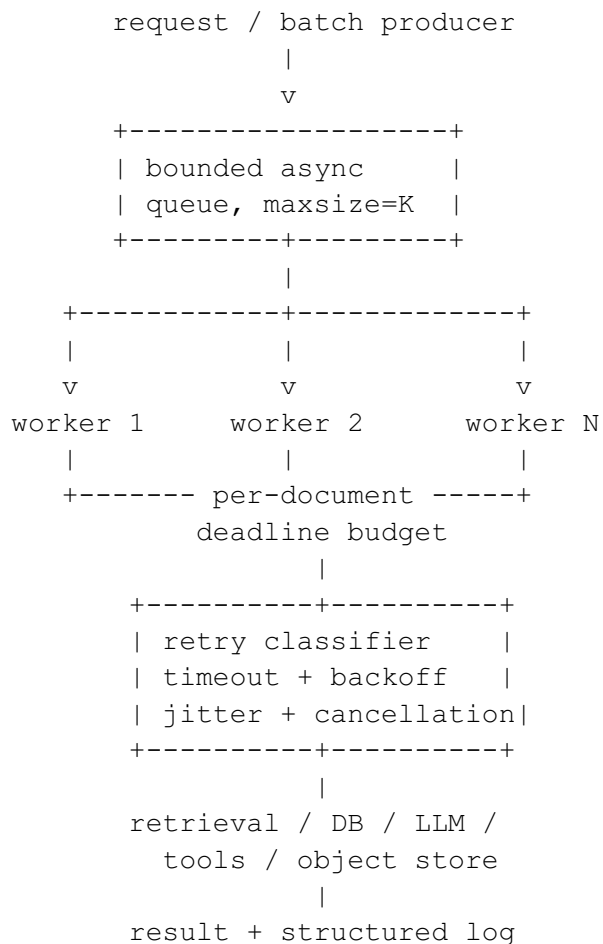
When an `asyncio.Queue` reaches its configured `maxsize`, `await queue.put()` blocks until a consumer removes an item. This creates backpressure naturally. `queue.join()` waits until every queued item has received a matching `task_done()`. ([Python documentation](#))

Preferred production pattern

- Fixed number of consumer tasks.

- Bounded input queue.
- One document processed by one consumer.
- Producer pauses when the queue is full.
- Downstream-specific semaphores when one dependency has a lower limit than the overall worker.

5. Architecture



6. Concurrent LLM, retrieval, database, and tool calls

Only independent operations should run concurrently.

```

async def answer_question(query: str, customer_id: str) -> dict:
    retrieval_task = asyncio.create_task(
        retriever.search(query)
    )
    profile_task = asyncio.create_task(
        customer_repository.get_profile(customer_id)
    )

    chunks, profile = await asyncio.gather(
        retrieval_task,

```

```
        profile_task,
    )

    # Generation depends on retrieval and profile, so it is sequential.
    return await llm.generate(
        query=query,
        context=chunks,
        customer_profile=profile,
    )
```

Good fan-out examples

- Vector retrieval and metadata lookup
- Independent tools such as pricing and inventory
- Queries against independent data sources
- Parallel model calls for an ensemble
- Parallel evaluation metrics

Incorrect fan-out examples

- Calling the LLM before retrieval completes
- Launching 500 tools when only two are relevant
- Performing concurrent writes to the same mutable entity
- Running ten retries concurrently for one failed request
- Exceeding the database connection pool with application concurrency

Downstream-specific limits

One global concurrency value is often insufficient:

```
llm_limit = asyncio.Semaphore(8)
database_limit = asyncio.Semaphore(20)
tool_limit = asyncio.Semaphore(5)
```

The limit should account for:

- Provider quotas
- Database pool size
- Per-request fan-out
- Number of application replicas
- Tail latency
- Cost
- Memory
- Rate limits

For example, 10 application replicas × 20 LLM calls per replica means the provider may see up to 200 concurrent requests.

7. Blocking work inside async code

This blocks the event loop:

```
async def handler():
    result = blocking_sdk.call()  # Bad
    return result
```

Other requests cannot make progress while the call runs.

For a blocking I/O library:

```
async def handler():
    result = await asyncio.to_thread(blocking_sdk.call)
    return result
```

Important limitation:

Cancelling the awaiting coroutine does not forcibly terminate arbitrary work already running inside the thread.

Therefore, configure timeouts in the underlying HTTP or database client as well. An application-level timeout alone may stop waiting while the underlying thread continues consuming resources.

For CPU-bound work:

```
from concurrent.futures import ProcessPoolExecutor

process_pool = ProcessPoolExecutor()

async def parse_large_document(data: bytes):
    loop = asyncio.get_running_loop()
    return await loop.run_in_executor(
        process_pool,
        parse_document_sync,
        data,
    )
```

Do not create a new thread or process pool per request. Pools should normally be application-scoped and closed during shutdown.

8. Cancellation

Cancellation is a control signal, not an ordinary processing failure.

When a caller disconnects, a server shuts down, or a parent deadline expires:

1. Cancel child tasks.
2. Release resources in `finally` or context managers.

3. Re-raise `CancelledError`.
4. Do not convert cancellation into a successful result.
5. Do not retry cancellation.

```
async def process():
    resource = await acquire_resource()
    try:
        return await perform_work(resource)
    except asyncio.CancelledError:
        await record_interruption()
        raise
    finally:
        await resource.close()
```

`asyncio.timeout()` cancels the current task internally when its deadline expires and exposes the outcome as `TimeoutError` outside the timeout context. Swallowing `CancelledError` can therefore interfere with timeout and structured-concurrency behavior. ([Python documentation](#))

9. Timeout budgets

A timeout should be treated as a budget, not independently reset at every layer.

Conceptual request budget:

Overall request budget:	8.0 seconds
authentication:	0.2
retrieval and metadata:	1.5
generation:	5.0
serialization/network:	0.3
safety margin:	1.0

A weak implementation gives every retry a fresh five-second timeout:

```
3 attempts × 5 seconds + backoff = much longer than caller deadline
```

A better implementation calculates one monotonic deadline:

```
deadline = loop.time() + total_budget
remaining = deadline - loop.time()
attempt_timeout = min(per_attempt_timeout, remaining)
```

Use a monotonic clock such as `loop.time()`, not wall-clock time, because wall-clock time can change due to clock synchronization or manual adjustment.

Timeout layers

- Client connection timeout
- Client read timeout

- Per-attempt timeout
- Per-document deadline
- Entire batch deadline
- API request deadline
- Queue visibility or lease timeout

The inner timeout should normally be shorter than the outer timeout so cleanup and response handling have time to complete.

10. Retry classification

Usually retryable

Failure	Condition
Connection reset	Remote service may recover
HTTP 429	Respect <code>Retry-After</code> where available
HTTP 502, 503, 504	Transient gateway or service failure
Temporary DNS/network error	Retry within total deadline
Database serialization conflict	Retry the complete transaction
Provider timeout	Only when the operation is safe to repeat

Usually permanent

Failure	Reason
Invalid request or schema	Same request will fail again
Unsupported document format	Requires data or code correction
Authentication or authorization failure	Blind retries create noise
Content-policy rejection	Same content is likely to be rejected
Missing required field	Caller must correct input
Programming error	Retrying hides a defect

Conditional

- HTTP 409 may represent either conflict or temporary contention.
- HTTP 404 may be eventually consistent or genuinely missing.
- HTTP 401 may be retryable once after credential refresh.
- A timeout may be unsafe to retry when the server could have completed a write.

Critical rule

Retrying a write is safe only when at least one is true:

- The operation is naturally idempotent.

- An idempotency key is used.
 - The database enforces a unique operation identifier.
 - The current operation status can be queried safely.
-

11. Exponential backoff and jitter

Basic exponential backoff:

```
delay = base_delay × 2^(attempt - 1)
```

Capped backoff:

```
cap = min(max_delay, base_delay × 2^(attempt - 1))
```

Full jitter:

```
delay = random(0, cap)
```

Without jitter, many replicas can retry at the same moments:

```
failure at t=0  
all retry at t=1  
all retry at t=3  
all retry at t=7
```

That synchronized retry pattern can prevent a recovering dependency from stabilizing.

Preventing retry storms

Use several controls together:

- Bounded concurrency
 - Jitter
 - Maximum attempts
 - Overall deadline
 - `Retry-After`
 - Circuit breaker
 - Global retry budget
 - Admission control
 - Idempotency
 - Dead-letter handling for background jobs
-

12. Testing strategy

Test type	Purpose	Example
Unit	One function or policy in isolation	Retry classifier, backoff calculation
Integration	Real integration with one dependency	Repository against temporary PostgreSQL
Contract	Verify boundary format and semantics	LLM adapter accepts current provider response
Component	Complete service with dependencies stubbed	API + worker + fake LLM
End-to-end	Whole deployed path	Upload → ingestion → retrieval → answer
Load/ resilience	Behavior under stress and failure	Provider latency, queue saturation, cancellation

Pytest techniques

Fixtures

Use fixtures for reusable setup:

```
@pytest.fixture
def document():
    return Document(...)
```

Parametrization

Use parametrization when the same behavior must hold across several inputs:

```
@pytest.mark.parametrize(
    "exception",
    [PermanentError("invalid"), ValueError("bug")],
)
def test_not_retried(exception):
    ...
```

Pytest supports fixture parametrization and `@pytest.mark.parametrize` for running a test with multiple argument sets. ([pytest](#))

Mocking

Prefer dependency injection:

```
worker = Worker(processor=fake_processor)
```

over globally patching an SDK.

Use `monkeypatch` when code depends on environment variables, module attributes, or difficult global dependencies. Pytest automatically restores monkeypatched changes after the test. ([pytest](#))

Deterministic async tests

Avoid:

- Real network calls
- Actual retry delays
- Random jitter
- Wall-clock assumptions
- Shared global clients
- Tests that depend on task scheduling order

Inject:

- Sleep function
 - Random-number function
 - Clock where necessary
 - External adapters
 - Failure scripts
-

13. Testing RAG and AI workflows

Ingestion

Test:

- Stable document identifiers
- Deterministic chunk boundaries
- Empty document handling
- Unsupported types
- Duplicate upload idempotency
- Partial processing cleanup
- Reprocessing after model-version change

Retrieval

Test:

- Expected documents appear for golden queries
- Tenant and authorization filters are enforced
- Deleted documents are excluded
- Metadata filtering
- Deduplication
- Empty retrieval
- Timeout behavior

Generation

Test:

- Output schema
- Required citations
- Empty-context behavior
- Provider error classification
- Token or response-size limits
- Cancellation
- Idempotency for tool side effects

Asynchronous workers

Test:

- At-least-once delivery duplicates
- Worker crash after external side effect but before acknowledgement
- Expired leases
- Poison messages
- Dead-letter routing
- Graceful shutdown
- Queue saturation
- Reordered completion

14. Structured logging and tracing

A useful request log contains stable fields:

```
{
  "event": "document_retry_scheduled",
  "level": "WARNING",
  "document_id": "doc-123",
  "correlation_id": "corr-456",
  "trace_id": "trace-789",
  "attempt": 2,
  "delay_s": 0.42,
  "error_type": "RetryableError"
}
```

Identifier roles

- **Correlation ID:** Connects logs belonging to one business request.
- **Trace ID:** Connects distributed spans across services.
- **Span ID:** Identifies one operation within a trace.
- **Job ID:** Identifies durable asynchronous work.
- **Idempotency key:** Identifies one logical side-effecting operation.

Python’s standard logging system supports contextual logging, including `LoggerAdapter`; structured systems commonly add the same context through adapters, filters, context variables, or `extra` fields. ([Python documentation](#))

Never log by default

- Raw document contents
- Complete prompts
- Authorization headers
- API keys
- Access tokens
- Personal financial information
- Embeddings
- Unredacted model responses
- Database connection strings

The strongest redaction strategy is not to emit sensitive data in the first place.

15. Common production concurrency failures

Check-then-act race

```
if not await repository.exists(request_id):  
    await repository.insert(request_id)
```

Two workers can both observe “not found” and both insert.

Use a unique database constraint and handle the conflict.

Lost update

```
balance = await read_balance()  
await write_balance(balance + amount)
```

Concurrent updates can overwrite one another.

Use transactions, atomic updates, row locks, or optimistic version checking.

Shared mutable state

```
current_request_id = None
```

A module-level value is shared across requests. Use request-local arguments or `contextvars`.

Resource leaks

Common causes:

- Client session created per request and never closed
- Unconsumed streaming response
- Task created but never awaited
- Semaphore acquired without `finally`
- Queue item retrieved without `task_done`
- Process or thread pool never shut down

Backpressure failure

Without backpressure:

```
incoming rate > processing rate
      ↓
unbounded queued tasks
      ↓
memory growth
      ↓
timeouts and retries
      ↓
more load
      ↓
service collapse
```

A bounded queue converts uncontrolled memory growth into waiting, rejection, or load shedding.

16. Practical task design

Thought process

We need a worker with these properties:

1. **Bounded memory:** Do not create one task per document.
2. **Bounded execution:** Exactly `N` consumers process documents.
3. **Per-document deadline:** Retries must share one total budget.
4. **Per-attempt timeout:** One slow provider call cannot consume the entire budget.
5. **Explicit retry classification:** Only timeout and `RetryableError` are retried.
6. **Jitter:** Prevent synchronized retries.
7. **Cancellation propagation:** External cancellation stops workers and is re-raised.
8. **Structured logs:** Include identifiers and attempt information.
9. **Sensitive-data protection:** Never log document content.
10. **Testability:** Inject processor, sleep, randomness, and logger.
11. **Stable output ordering:** Results correspond to input order, even though completion is concurrent.

Correctness conditions

For N configured consumers:

- No more than N documents are actively processed.
- Every enqueued item receives exactly one `task_done()`.
- Every input receives exactly one terminal result unless the batch is cancelled.
- A permanent failure performs one attempt.
- A retryable failure performs at most `max_attempts`.
- A retry never begins after the document deadline expires.
- `CancelledError` is never converted into an ordinary failure.
- Raw document content never appears in logs.

Pseudocode

```
function run(documents):
    create bounded queue
    create result array matching input size
    start N consumer tasks

    for every document:
        await queue.put(index, document)
        # blocks when queue is full: backpressure

    put one stop marker for every consumer
    wait until queue has been fully processed
    await all consumers

    if batch is cancelled:
        cancel all consumers
        await their cleanup
        re-raise cancellation

    return results in input order

function process_with_retry(document):
    deadline = monotonic_time + total_document_timeout

    for attempt from 1 to max_attempts:
        remaining = deadline - monotonic_time

        if remaining <= 0:
            return timeout result

        attempt_timeout = minimum(per_attempt_timeout, remaining)

        try:
            process document under attempt_timeout
            return success

        catch cancellation:
```



```

        log cancellation
        re-raise

    catch permanent error:
        return permanent failure

    catch timeout or retryable error:
        if final attempt:
            return timeout or retry-exhausted

        delay = random value between 0 and exponential cap

        if delay exceeds remaining budget:
            return timeout

        await delay

    catch unknown exception:
        fail closed as permanent failure

```

17. Implementation

document_worker.py

```

from __future__ import annotations

import asyncio
import json
import logging
import random
import time
from collections.abc import Awaitable, Callable, Sequence
from dataclasses import dataclass
from typing import Any, Protocol

class RetryableError(Exception):
    """Transient failure that may succeed on a later attempt."""

class PermanentError(Exception):
    """Failure that should not be retried."""

class AsyncDocumentProcessor(Protocol):
    async def process(self, document: "Document") -> dict[str, Any]:
        ...

@dataclass(frozen=True, slots=True)
class Document:
    document_id: str

```

```

    content: str
    correlation_id: str
    trace_id: str | None = None

@dataclass(frozen=True, slots=True)
class RetryPolicy:
    max_attempts: int = 3
    base_delay_s: float = 0.1
    max_delay_s: float = 2.0

    def __post_init__(self) -> None:
        if self.max_attempts < 1:
            raise ValueError("max_attempts must be at least 1")
        if self.base_delay_s < 0 or self.max_delay_s < 0:
            raise ValueError("retry delays must be non-negative")
        if self.base_delay_s > self.max_delay_s:
            raise ValueError(
                "base_delay_s cannot exceed max_delay_s"
            )

@dataclass(frozen=True, slots=True)
class ProcessResult:
    document_id: str
    status: str
    attempts: int
    output: dict[str, Any] | None = None
    error_type: str | None = None
    error_message: str | None = None

class JsonFormatter(logging.Formatter):
    """Minimal structured formatter with defensive redaction."""

    RESERVED = {
        "name", "msg", "args", "levelname", "levelno",
        "pathname", "filename", "module", "exc_info",
        "exc_text", "stack_info", "lineno", "funcName",
        "created", "msecs", "relativeCreated", "thread",
        "threadName", "processName", "process", "taskName",
    }

    SENSITIVE_FRAGMENTS = (
        "authorization",
        "api_key",
        "apikey",
        "token",
        "password",
        "secret",
        "content",
        "document_text",
    )

```

```

@classmethod
def redact(cls, value: Any, key: str = "") -> Any:
    if any(
        fragment in key.lower()
        for fragment in cls.SENSITIVE_FRAGMENTS
    ):
        return "[REDACTED]"

    if isinstance(value, dict):
        return {
            child_key: cls.redact(child_value, child_key)
            for child_key, child_value in value.items()
        }

    if isinstance(value, (list, tuple)):
        return [cls.redact(item) for item in value]

    return value

def format(self, record: logging.LogRecord) -> str:
    payload: dict[str, Any] = {
        "timestamp": time.strftime(
            "%Y-%m-%dT%H:%M:%SZ",
            time.gmtime(record.created),
        ),
        "level": record.levelname,
        "event": getattr(
            record,
            "event",
            record.getMessage(),
        ),
    }

    for key, value in record.__dict__.items():
        if key not in self.RESERVED and key not in payload:
            payload[key] = self.redact(value, key)

    if record.exc_info:
        payload["exception"] = self.formatException(
            record.exc_info
        )

    return json.dumps(
        payload,
        default=str,
        separators=(",", ":"),
    )

def build_json_logger(
    name: str = "document-worker",
) -> logging.Logger:
    logger = logging.getLogger(name)

```

```

if not logger.handlers:
    handler = logging.StreamHandler()
    handler.setFormatter(JsonFormatter())
    logger.addHandler(handler)

logger.setLevel(logging.INFO)
logger.propagate = False
return logger

SleepFunction = Callable[[float], Awaitable[None]]
RandomFunction = Callable[[], float]

class BoundedDocumentWorker:
    def __init__(
        self,
        processor: AsyncDocumentProcessor,
        *,
        concurrency: int = 4,
        queue_size: int = 16,
        attempt_timeout_s: float = 10.0,
        document_timeout_s: float = 30.0,
        retry_policy: RetryPolicy | None = None,
        sleep: SleepFunction = asyncio.sleep,
        random_fn: RandomFunction = random.random,
        logger: logging.Logger | None = None,
    ) -> None:
        if concurrency < 1:
            raise ValueError(
                "concurrency must be at least 1"
            )
        if queue_size < 1:
            raise ValueError(
                "queue_size must be at least 1"
            )
        if (
            attempt_timeout_s <= 0
            or document_timeout_s <= 0
        ):
            raise ValueError("timeouts must be positive")

        self._processor = processor
        self._concurrency = concurrency
        self._queue_size = queue_size
        self._attempt_timeout_s = attempt_timeout_s
        self._document_timeout_s = document_timeout_s
        self._retry_policy = (
            retry_policy or RetryPolicy()
        )
        self._sleep = sleep
        self._random_fn = random_fn
        self._logger = logger or build_json_logger()

```

```

def _log(
    self,
    level: int,
    event: str,
    document: Document | None = None,
    **fields: Any,
) -> None:
    context: dict[str, Any] = {
        "event": event,
        **fields,
    }

    if document is not None:
        context.update(
            document_id=document.document_id,
            correlation_id=document.correlation_id,
            trace_id=document.trace_id,
        )

    self._logger.log(
        level,
        event,
        extra=context,
    )

def _full_jitter_delay(
    self,
    failed_attempt: int,
) -> float:
    exponential_cap = (
        self._retry_policy.base_delay_s
        * (2 ** (failed_attempt - 1))
    )
    capped_delay = min(
        self._retry_policy.max_delay_s,
        exponential_cap,
    )
    return self._random_fn() * capped_delay

async def _process_with_retry(
    self,
    document: Document,
) -> ProcessResult:
    loop = asyncio.get_running_loop()
    deadline = (
        loop.time() + self._document_timeout_s
    )
    last_error: BaseException | None = None

    for attempt in range(
        1,
        self._retry_policy.max_attempts + 1,
    ):
        remaining_budget = deadline - loop.time()

```

```

if remaining_budget <= 0:
    return self._timeout_result(
        document,
        attempts=attempt - 1,
        error=last_error,
    )

attempt_timeout = min(
    self._attempt_timeout_s,
    remaining_budget,
)

self._log(
    logging.INFO,
    "document_attempt_started",
    document,
    attempt=attempt,
)

try:
    async with asyncio.timeout(
        attempt_timeout
    ):
        output = (
            await self._processor.process(
                document
            )
        )

except asyncio.CancelledError:
    self._log(
        logging.WARNING,
        "document_cancelled",
        document,
        attempt=attempt,
    )
    raise

except TimeoutError as exc:
    last_error = exc
    error_type = "TimeoutError"

except RetryableError as exc:
    last_error = exc
    error_type = type(exc).__name__

except PermanentError as exc:
    self._log(
        logging.ERROR,
        "document_failed_permanently",
        document,
        attempt=attempt,
        error_type=type(exc).__name__,
    )

```

```

    )

    return ProcessResult(
        document_id=document.document_id,
        status="permanent_failure",
        attempts=attempt,
        error_type=type(exc).__name__,
        error_message=str(exc),
    )

except Exception as exc:
    # Unknown failures are not assumed transient.
    self._log(
        logging.ERROR,
        "document_failed_unclassified",
        document,
        attempt=attempt,
        error_type=type(exc).__name__,
    )

    return ProcessResult(
        document_id=document.document_id,
        status="permanent_failure",
        attempts=attempt,
        error_type=type(exc).__name__,
        error_message=str(exc),
    )

else:
    self._log(
        logging.INFO,
        "document_succeeded",
        document,
        attempt=attempt,
    )

    return ProcessResult(
        document_id=document.document_id,
        status="success",
        attempts=attempt,
        output=output,
    )

if (
    attempt
    == self._retry_policy.max_attempts
):
    status = (
        "timeout"
        if isinstance(last_error, TimeoutError)
        else "retry_exhausted"
    )

    self._log(

```

```

        logging.ERROR,
        "document_attempts_exhausted",
        document,
        attempt=attempt,
        status=status,
        error_type=error_type,
    )

    return ProcessResult(
        document_id=document.document_id,
        status=status,
        attempts=attempt,
        error_type=error_type,
        error_message=str(last_error),
    )

    delay = self._full_jitter_delay(attempt)
    remaining_after_attempt = (
        deadline - loop.time()
    )

    if delay >= remaining_after_attempt:
        return self._timeout_result(
            document,
            attempts=attempt,
            error=last_error,
        )

    self._log(
        logging.WARNING,
        "document_retry_scheduled",
        document,
        attempt=attempt,
        delay_s=round(delay, 6),
        error_type=error_type,
    )

    await self._sleep(delay)

    raise AssertionError("unreachable")

def _timeout_result(
    self,
    document: Document,
    *,
    attempts: int,
    error: BaseException | None,
) -> ProcessResult:
    error_type = (
        type(error).__name__
        if error is not None
        else "TimeoutError"
    )

```



```

self._log(
    logging.ERROR,
    "document_deadline_exceeded",
    document,
    attempt=attempts,
    error_type=error_type,
)

return ProcessResult(
    document_id=document.document_id,
    status="timeout",
    attempts=attempts,
    error_type=error_type,
    error_message=(
        str(error)
        if error is not None
        else "document deadline exceeded"
    ),
)

async def run(
    self,
    documents: Sequence[Document],
) -> list[ProcessResult]:
    queue: asyncio.Queue[
        tuple[int, Document] | None
    ] = asyncio.Queue(
        maxsize=self._queue_size
    )

    results: list[ProcessResult | None] = [
        None
    ] * len(documents)

    async def consume(worker_id: int) -> None:
        while True:
            item = await queue.get()

            try:
                if item is None:
                    return

                index, document = item
                results[index] = (
                    await self._process_with_retry(
                        document
                    )
                )
            finally:
                queue.task_done()

    workers = [
        asyncio.create_task(
            consume(worker_id),

```

```

        name=f"document-worker-{worker_id}",
    )
    for worker_id in range(
        self._concurrency
    )
]

self._log(
    logging.INFO,
    "batch_started",
    document_count=len(documents),
    concurrency=self._concurrency,
    queue_size=self._queue_size,
)

try:
    for index, document in enumerate(
        documents
    ):
        await queue.put(
            (index, document)
        )

    # One stop marker per consumer.
    for _ in workers:
        await queue.put(None)

    await queue.join()
    await asyncio.gather(*workers)

except asyncio.CancelledError:
    self._log(
        logging.WARNING,
        "batch_cancelled",
    )

    for task in workers:
        task.cancel()

    await asyncio.gather(
        *workers,
        return_exceptions=True,
    )
    raise

except BaseException:
    for task in workers:
        task.cancel()

    await asyncio.gather(
        *workers,
        return_exceptions=True,
    )
    raise

```

```

        if any(result is None for result in results):
            raise RuntimeError(
                "batch finished with missing results"
            )

        self._log(
            logging.INFO,
            "batch_finished",
            document_count=len(documents),
        )

        return [
            result
            for result in results
            if result is not None
        ]

```

18. Pytest suite

The tests use `asyncio.run()`, so `pytest-asyncio` is not required.

`test_document_worker.py`

```

import asyncio
from collections import deque

import pytest

from document_worker import (
    BoundedDocumentWorker,
    Document,
    PermanentError,
    RetryableError,
    RetryPolicy,
)

class ScriptedProcessor:
    """Returns or raises outcomes in a predefined order."""

    def __init__(self, outcomes):
        self.outcomes = deque(outcomes)
        self.calls = 0

    async def process(self, document):
        self.calls += 1
        outcome = self.outcomes.popleft()

        if outcome == "slow":
            await asyncio.sleep(0.05)
            return {"should_not": "complete"}

```

```

        if isinstance(outcome, BaseException):
            raise outcome

        return outcome

@pytest.fixture
def document():
    return Document(
        document_id="doc-1",
        content="confidential document body",
        correlation_id="corr-123",
        trace_id="trace-456",
    )

def make_worker(
    processor,
    *,
    max_attempts=3,
    attempt_timeout=0.02,
):
    async def no_sleep(_delay):
        # Removes real retry waiting from tests.
        return None

    return BoundedDocumentWorker(
        processor,
        concurrency=2,
        queue_size=2,
        attempt_timeout_s=attempt_timeout,
        document_timeout_s=0.2,
        retry_policy=RetryPolicy(
            max_attempts=max_attempts,
            base_delay_s=0.001,
            max_delay_s=0.01,
        ),
        sleep=no_sleep,
        # Deterministic jitter.
        random_fn=lambda: 0.0,
    )

def test_success(document):
    processor = ScriptedProcessor([
        {"chunks": 4}
    ])

    result = asyncio.run(
        make_worker(processor).run([document])
    )[0]

    assert result.status == "success"

```

```

    assert result.attempts == 1
    assert result.output == {"chunks": 4}
    assert processor.calls == 1

def test_timeout(document):
    processor = ScriptedProcessor(["slow"])

    result = asyncio.run(
        make_worker(
            processor,
            max_attempts=1,
            attempt_timeout=0.001,
        ).run([document])
    )[0]

    assert result.status == "timeout"
    assert result.attempts == 1
    assert result.error_type == "TimeoutError"
    assert processor.calls == 1

def test_retryable_failure_then_success(document):
    processor = ScriptedProcessor([
        RetryableError(
            "provider temporarily returned 503"
        ),
        {"chunks": 4},
    ])

    result = asyncio.run(
        make_worker(processor).run([document])
    )[0]

    assert result.status == "success"
    assert result.attempts == 2
    assert processor.calls == 2

@pytest.mark.parametrize(
    "failure",
    [
        PermanentError(
            "unsupported file type"
        ),
        ValueError(
            "unexpected schema bug"
        ),
    ],
)
def test_permanent_failure_is_not_retried(
    document,
    failure,
):

```

```
processor = ScriptedProcessor([failure])

result = asyncio.run(
    make_worker(processor).run([document])
)[0]

assert result.status == "permanent_failure"
assert result.attempts == 1
assert processor.calls == 1

def test_batch_cancellation_propagates(document):
    processor = ScriptedProcessor(["slow"])

    worker = make_worker(
        processor,
        max_attempts=1,
        attempt_timeout=1.0,
    )

    async def scenario():
        batch_task = asyncio.create_task(
            worker.run([document])
        )

        # Allow the batch task to start.
        await asyncio.sleep(0)

        batch_task.cancel()

        with pytest.raises(
            asyncio.CancelledError
        ):
            await batch_task

    asyncio.run(scenario())
```

Run:

```
python -m pip install pytest
pytest -q
```

Validated result:

```
6 passed
```

19. Why the non-obvious parts matter

The queue is bounded

`queue_size` limits waiting work. When full, the producer pauses instead of creating more in-memory tasks.

Consumers are fixed

Only `concurrency` consumer tasks exist, so active document processing is bounded.

Retries share one deadline

The deadline is created once outside the retry loop. A new attempt cannot reset the total budget.

Attempt timeout is capped by remaining time

```
attempt_timeout = min(  
    configured_attempt_timeout,  
    remaining_document_budget,  
)
```

This prevents the inner operation from outliving the outer deadline.

Cancellation is re-raised

Cancellation causes cleanup but is not classified as timeout, permanent failure, or retryable failure.

Unknown errors fail closed

Blindly retrying every exception can retry programming defects, invalid data, and deterministic schema errors.

Production systems can replace this with an explicit classifier:

```
RetryDecision = Literal[  
    "retry",  
    "permanent",  
    "refresh_credentials_then_retry",  
]
```

Sleep and randomness are injected

Tests do not wait for real backoff and do not depend on random numbers.

Results preserve input order

Workers may finish out of order, but each result is written into the input index.

Content is excluded from logs

The worker logs `document_id`, not `document.content`. The formatter also provides defensive redaction, but omission remains the primary control.

20. Production extensions

For a real AI ingestion platform, extend this design with:

- Provider-specific exception mapping
- `Retry-After` handling
- Metrics for queue depth, active workers, attempts, timeouts, and latency
- OpenTelemetry spans
- Circuit breaker
- Per-provider semaphore
- Durable message queue
- Dead-letter queue
- Idempotency table
- Transactional outbox
- Graceful shutdown deadline
- Health and readiness checks
- Maximum input size
- Tenant-level fairness
- Priority queues
- Load shedding

For durable jobs, the in-process queue should be replaced or preceded by a persistent broker. An in-memory queue provides concurrency control and backpressure, but work is lost if the process terminates.

21. Senior interview questions

Why not use `asyncio.gather` over all documents?

Because `gather` coordinates tasks but does not bound task creation. For a large input set, it can allocate enormous numbers of pending tasks. A fixed worker pool over a bounded queue limits both active work and buffered work.

Why not retry all exceptions?

Some failures are deterministic: invalid input, authentication failure, policy rejection, and code defects. Retrying them wastes capacity and can hide defects.

Why use both per-attempt and total timeouts?

The per-attempt timeout limits one external call. The total timeout prevents retries and backoff from exceeding the caller's overall deadline.

Why is cancellation different from failure?

Cancellation communicates that the caller or parent scope no longer wants the work. Converting it into a normal result lets unwanted work continue and breaks shutdown and deadline propagation.

Can async improve CPU-heavy model inference?

Not directly. Async can prevent request threads from blocking while awaiting an external inference server. Local CPU-heavy Python should use processes, native libraries that release the GIL, or a separate inference service.

How do you prevent duplicate processing?

Use a stable job or idempotency key backed by a database uniqueness constraint. Do not rely on an in-memory “already processed” check.

How do you choose concurrency?

Start with downstream limits: database pool size, provider quotas, replica count, memory, and request fan-out. Then load test and tune based on saturation, throughput, error rate, and tail latency.

What is the biggest async mistake in Python services?

Calling blocking code from the event-loop thread. One blocking SDK call can stall every coroutine handled by that event loop.

Day 4 final checklist

You should now be able to explain and implement:

- I/O-bound versus CPU-bound execution
- Event loops, coroutines, and tasks
- `gather`, semaphores, and bounded queues
- Threads versus processes under the GIL
- Concurrent retrieval, database, tool, and LLM calls
- Deadline propagation
- Retry classification, exponential backoff, and jitter
- Cancellation-safe cleanup
- Deterministic pytest tests
- Structured logs with correlation and trace IDs
- Idempotency, race conditions, resource management, and backpressure

Day 4 DSA — Two Pointers

Beginner-friendly summary

The **two-pointers technique** uses two indexes that move through an array or string according to a rule.

Instead of checking every pair or repeatedly scanning the same elements, the pointers eliminate impossible choices as they move.

Typical result:

- Brute force: $O(n^2)$ or $O(n^3)$
- Two pointers: often $O(n)$ after sorting, or $O(n^2)$ for problems involving three elements

The most important question is:

| After comparing the current elements, can I safely decide which pointer must move?

1. Main two-pointer patterns

Pattern	Pointer movement	Typical use
Opposite-direction	One starts left, one starts right	Pair sum, palindrome, container area
Same-direction	Both move left to right	Remove duplicates, compact arrays, subsequences
Sliding-window style	Right expands, left contracts	Subarrays and substrings
Partitioning	Pointers rearrange elements into regions	Move zeros, sort colors, quicksort
Sorted-input search	Move based on value comparison	Two Sum II, 3Sum, closest sum

2. Recognition signals

Consider two pointers when you see one or more of these signals.

Signal 1: The input is sorted

Examples:

- Find two numbers with a target sum
- Find the closest pair
- Remove duplicates
- Find triplets with a target value

Sorted order lets pointer movement eliminate many possibilities.

For a sorted array:

small values	large values
↓	↓
[1, 2, 4, 6, 8, 11, 15]	
L	R

If `array[L] + array[R]` is too small, moving `R` left would make the sum even smaller. Therefore, only `L` should move right.

Signal 2: The problem asks about pairs or triplets

Keywords include:

- Pair
 - Triplet
 - Sum
 - Difference
 - Closest
 - Maximum area
 - Compare elements from both ends
-

Signal 3: You are repeatedly scanning remaining elements

A brute-force solution may contain:

```
for i in range(n):  
    for j in range(i + 1, n):  
        ...
```

Ask whether ordering allows one pointer movement to discard an entire group of pairs.

Signal 4: Elements must be modified in place

Examples:

- Remove duplicates
- Move zeros
- Partition positive and negative values
- Keep valid elements at the front

This commonly uses:

- A **read pointer** to inspect elements
 - A **write pointer** to store valid elements
-

Signal 5: You compare symmetrical positions

Examples:

- Palindrome validation
- Reverse an array
- Compare first and last characters
- Two-ended search

3. Opposite-direction pointers

One pointer begins at the start and another at the end.

```
left = 0
right = len(values) - 1

while left < right:
    ...
```

Common movement rule

For a sorted pair-sum problem:

```
current_sum = values[left] + values[right]

if current_sum < target:
    left += 1
elif current_sum > target:
    right -= 1
else:
    return True
```

Why this works

Suppose the sum is too small.

Because the array is sorted:

- Moving the right pointer left gives a value that is equal or smaller.
- That cannot increase the sum.
- Therefore, the left pointer must move right.

This is the correctness argument—not merely a coding trick.

4. Same-direction pointers

Both pointers move from left to right, but they have different responsibilities.

Example: remove duplicates from a sorted array

```
read pointer:  inspects each value
write pointer: marks where the next unique value belongs
```

```
def remove_duplicates(values: list[int]) -> int:
    if not values:
        return 0

    write = 1
```

```

for read in range(1, len(values)):
    if values[read] != values[write - 1]:
        values[write] = values[read]
        write += 1

return write

```

Invariant

Before each iteration:

```
values[0:write]
```

contains all unique values found so far.

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

5. Partitioning pointers

Partitioning divides an array into logical regions.

Example: move zeros to the end

```

def move_zeroes(values: list[int]) -> None:
    write = 0

    for read in range(len(values)):
        if values[read] != 0:
            values[write], values[read] = (
                values[read],
                values[write],
            )
            write += 1

```

During execution:

```

[ non-zero processed | zeros/unknown | unprocessed ]
           write           read

```

Correctness invariant

Before each iteration:

- Everything before `write` is a processed non-zero value.
- Everything between `write` and `read` consists of zeros or displaced values.
- Everything after `read` has not yet been examined.

6. Medium problem — 3Sum

Problem statement

Given an integer array `nums`, return all unique triplets:

```
[nums[i], nums[j], nums[k]]
```

such that:

```
i != j  
j != k  
i != k
```

and:

```
nums[i] + nums[j] + nums[k] == 0
```

The returned triplets must not contain duplicates.

Example

```
Input:  
[-1, 0, 1, 2, -1, -4]  
  
Output:  
[[-1, -1, 2], [-1, 0, 1]]
```

7. Recognition signals

This problem strongly suggests sorting plus two pointers because:

1. It asks for **triplets** satisfying a target sum.
2. Duplicate triplets must be avoided.
3. Once one number is fixed, the remaining problem becomes a two-number target-sum problem.
4. Sorted order tells us which pointer to move.
5. Sorted duplicate values can be skipped efficiently.

The transformation is:

```
a + b + c = 0
```

Fix `a`:

```
b + c = -a
```

Now solve a two-sum problem using opposite-direction pointers.

8. Brute-force reasoning

Basic idea

Try every possible triplet.

```
for i in range(n):
    for j in range(i + 1, n):
        for k in range(j + 1, n):
            if nums[i] + nums[j] + nums[k] == 0:
                ...
```

Duplicate problem

Different index combinations may produce the same values.

For:

```
[-1, 0, 1, 2, -1, -4]
```

both -1 occurrences may produce:

```
[-1, 0, 1]
```

We could place sorted triplets into a set:

```
triplets.add(tuple(sorted((nums[i], nums[j], nums[k]))))
```

Brute-force complexity

- Number of triplets: approximately $n^3 / 6$
- Time: $O(n^3)$
- Extra space: $O(m)$ for unique results

Here, m is the number of result triplets.

Why brute force is inadequate

For $n = 3,000$:

```
 $3,000^3 = 27,000,000,000$ 
```

The exact number of combinations is smaller, but still impractical.

9. Optimized reasoning

Step 1: Sort the array

```
Original:  
[-1, 0, 1, 2, -1, -4]  
  
Sorted:  
[-4, -1, -1, 0, 1, 2]
```

Sorting provides:

- Monotonic pointer movement
 - Easy duplicate skipping
 - Early termination opportunities
-

Step 2: Fix the first number

For every index `i`, treat `nums[i]` as the first number.

```
fixed = nums[i]
```

Then search for two numbers to the right whose sum equals:

```
-nums[i]
```

Step 3: Place two pointers

```
left = i + 1  
right = len(nums) - 1
```

Current triplet:

```
nums[i] + nums[left] + nums[right]
```

Step 4: Move based on the sum

Sum is too small

```
if current_sum < 0:  
    left += 1
```

We need a larger sum. Since the array is sorted, moving `left` right increases or preserves its value.

Sum is too large

```
elif current_sum > 0:
    right -= 1
```

We need a smaller sum. Moving `right` left decreases or preserves its value.

Sum equals zero

Record the triplet, then move both pointers and skip duplicates.

10. Duplicate handling

Duplicate handling is the most important non-obvious part of `3Sum`.

Skip duplicate fixed values

After processing one `-1`, processing another identical `-1` as the fixed value would produce the same triplets.

```
if i > 0 and nums[i] == nums[i - 1]:
    continue
```

Skip duplicate left and right values

After finding a valid triplet:

```
left += 1
right -= 1
```

Then skip repeated values:

```
while left < right and nums[left] == nums[left - 1]:
    left += 1

while left < right and nums[right] == nums[right + 1]:
    right -= 1
```

We skip duplicates only **after recording a valid triplet**, because repeating the same left or right value would reproduce the same value combination.

11. Early stopping

Because the array is sorted, once the fixed value becomes positive:

```
if nums[i] > 0:
    break
```

all values to its right are also positive.

Therefore:

```
positive + positive + positive > 0
```

No zero-sum triplet remains possible.

This optimization is correct specifically because the target is zero and the array is sorted.

12. Pseudocode

```
sort nums

create empty results

for each index i from 0 to n - 3:
    if nums[i] is greater than zero:
        stop

    if nums[i] equals the previous fixed value:
        skip it

    left = i + 1
    right = n - 1

    while left is before right:
        total = nums[i] + nums[left] + nums[right]

        if total is less than zero:
            move left rightward

        else if total is greater than zero:
            move right leftward

        else:
            add the triplet to results
            move both pointers

            skip duplicate left values
            skip duplicate right values

return results
```

13. Python solution

```
def three_sum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    triplets: list[list[int]] = []

    for i in range(len(nums) - 2):
        # Once the smallest value in the triplet is positive,
```

```
# the total can no longer be zero.
if nums[i] > 0:
    break

# Avoid repeating the same fixed first value.
if i > 0 and nums[i] == nums[i - 1]:
    continue

left = i + 1
right = len(nums) - 1

while left < right:
    total = nums[i] + nums[left] + nums[right]

    if total < 0:
        left += 1

    elif total > 0:
        right -= 1

    else:
        triplets.append(
            [nums[i], nums[left], nums[right]]
        )

        left += 1
        right -= 1

        # Avoid producing the same triplet again.
        while (
            left < right
            and nums[left] == nums[left - 1]
        ):
            left += 1

        while (
            left < right
            and nums[right] == nums[right + 1]
        ):
            right -= 1

return triplets
```

14. Dry run

Input:

```
[-1, 0, 1, 2, -1, -4]
```

After sorting:

```
[-4, -1, -1, 0, 1, 2]
```

Fixed value: -4

```
i = 0
left = -1
right = 2

-4 + -1 + 2 = -3
```

Too small, so move `left`.

All subsequent sums remain below zero, so no result is found for -4.

Fixed value: first -1

```
[-4, -1, -1, 0, 1, 2]
      i   L           R
```

```
-1 + -1 + 2 = 0
```

Record:

```
[-1, -1, 2]
```

Move both pointers:

```
left  → 0
right → 1
```

Next:

```
-1 + 0 + 1 = 0
```

Record:

```
[-1, 0, 1]
```

Fixed value: second -1

It equals the previous fixed value, so skip it.

This prevents duplicate triplets.

Final result

```
[
    [-1, -1, 2],
    [-1, 0, 1],
]
```

15. Correctness reasoning

A senior-level explanation should establish why the algorithm cannot miss a valid triplet.

Invariant

For a fixed `i`, all candidate pairs lie between `left` and `right`.

At each step:

- When the total is too small, every pair using the current `left` and any index at or before `right` is also too small or no larger.
- Therefore, the current `left` cannot participate in a valid zero-sum pair for this fixed value.
- Moving `left` is safe.

Similarly:

- When the total is too large, every pair using the current `right` and any index at or after `left` is also too large or no smaller.
- Therefore, moving `right` is safe.

Each movement eliminates only combinations that cannot satisfy the target.

Duplicate skipping removes repeated value combinations, not distinct triplets.

16. Edge cases

Empty input

```
three_sum([])
# []
```

Fewer than three values

```
three_sum([1, -1])
# []
```

The loop does not execute.

All positive

```
three_sum([1, 2, 3])  
# []
```

The early positive check stops immediately.

All negative

```
three_sum([-5, -3, -1])  
# []
```

All sums remain negative.

All zeros

```
three_sum([0, 0, 0, 0])  
# [[0, 0, 0]]
```

Duplicate skipping prevents repeated `[0, 0, 0]`.

Many duplicates

```
three_sum([-2, -2, 0, 0, 2, 2])  
# [[-2, 0, 2]]
```

No valid triplet

```
three_sum([1, 2, -2, -1])  
# []
```

Input mutation

The function calls:

```
nums.sort()
```

Therefore, it modifies the caller's list.

To preserve the original input:

```
nums = sorted(nums)
```

That creates another list and requires $O(n)$ additional space.

17. Complexity

Time complexity

Sorting:

$O(n \log n)$

Outer loop:

$O(n)$

For each fixed value, the two pointers traverse the remaining array once:

$O(n)$

Total:

$O(n^2)$

Since $O(n^2)$ dominates $O(n \log n)$:

Final time complexity: $O(n^2)$

Space complexity

Ignoring the returned output:

- Python sorting may use implementation-dependent temporary memory.
- The two-pointer logic itself uses $O(1)$ auxiliary variables.

Interview answer:

The algorithm uses $O(1)$ two-pointer auxiliary space if sorting memory and the output are excluded. In Python, the actual sort can consume additional temporary memory, so I would state that implementation detail when strict space analysis matters.

The result itself requires:

$O(m)$

where m is the number of returned triplets.

18. Common mistakes

Mistake 1: Using a set instead of proper duplicate handling

A set may produce correct results but:

- Uses additional memory
- Hides incomplete reasoning
- Requires tuple conversion
- May generate many duplicate candidates before removing them

Skipping duplicates directly is more efficient and demonstrates mastery.

Mistake 2: Moving only one pointer after finding a triplet

After a valid triplet:

```
left += 1
right -= 1
```

Both should move because retaining either current value cannot create a new unique triplet with the other pointer moving through duplicates.

Mistake 3: Skipping duplicates before evaluating the first occurrence

Incorrect duplicate logic can skip valid triplets.

For the fixed pointer, skip only when:

```
i > 0 and nums[i] == nums[i - 1]
```

This preserves the first occurrence.

Mistake 4: Forgetting that sorting mutates the input

Use:

```
nums = sorted(nums)
```

when mutation is not allowed.

Mistake 5: Using `left <= right`

A triplet requires three distinct indexes.

The two inner pointers must satisfy:

```
left < right
```


When `left == right`, the same array element would be used twice.

Mistake 6: Moving pointers without using sorted-order reasoning

Two pointers are not automatically valid just because an array contains numbers. The movement must eliminate impossible choices.

Without sorted order, moving left or right based on the sum is not logically justified.

19. Independent practice version

Before reviewing the solution again, try implementing it from this reduced prompt:

```
Given nums, return every unique triplet whose sum is zero.
```

Constraints:

- Do not use three nested loops.
- Do not use a set to remove duplicate result triplets.
- Preserve distinct indexes.
- Target $O(n^2)$ time.

Think about:

1. Why sorting helps.
2. What value should be fixed.
3. Which direction each pointer moves.
4. Where duplicates must be skipped.
5. When the outer loop can stop early.

Test cases:

```
assert three_sum(
    [-1, 0, 1, 2, -1, -4]
) == [
    [-1, -1, 2],
    [-1, 0, 1],
]

assert three_sum([0, 0, 0, 0]) == [
    [0, 0, 0]
]

assert three_sum([1, 2, 3]) == []

assert three_sum([]) == []

assert three_sum(
    [-2, -2, 0, 0, 2, 2]
) == [
    [-2, 0, 2]
]
```

20. Interview-ready explanation

I first sort the array. I then fix one number and reduce the remaining problem to finding two numbers whose sum equals the negative of the fixed number. I use a left and right pointer because the sorted order tells me how to move: if the total is too small, I move left to increase it; if it is too large, I move right to decrease it. I skip repeated fixed, left, and right values to avoid duplicate triplets. Sorting costs $O(n \log n)$, and the fixed-number plus linear two-pointer scan costs $O(n^2)$ overall.